Graph Neural Networks (GNN)

Applied to the Travelling Salesman Problem (TSP)

April 5, 2026

Abstract

This document presents the application of Graph Neural Networks (GNNs) to the Travelling Salesman Problem (TSP). After a formal definition and a complete mathematical description of the TSP-GNN architecture (Joshi *et al.*, 2019), we describe the supervised training pipeline, complexity analysis, strengths and limitations of the approach. The benchmark covers sizes $n \in \{1, 10, 50, 100, 200\}$ and reports success rate, non-detection rate, and inference time metrics. The scalability ceiling — where the dense $O(n^2 d)$ edge embedding becomes infeasible — is analysed and quantified.

Contents

1 Definition **2**

2 Formal Description **2**
- 2.1 Graph representation . . . 2
- 2.2 Message Passing Neural Network (MPNN) framework . . . 3
- 2.3 TSP-GNN architecture (Joshi *et al.*, 2019) . . . 3
- 2.4 Training objective . . . 4
- 2.5 Complexity . . . 4

3 Architecture / Algorithm Steps **5**

4 Strengths and Limitations **5**

5 Use Case Explanation **6**

6 Code Implementation **7**

7 Benchmark **8**
- 7.1 Scalability analysis for large instances . . . 10

8 Experimental Analysis **10**

1 Definition

Graph Neural Networks (GNNs) are a family of deep learning models designed to operate directly on graph-structured data. Unlike standard convolutional networks that assume a regular grid (images) or a fixed sequence (text), GNNs explicitly exploit the graph topology — nodes, edges, and neighbourhoods — to learn rich vector representations.

Origin

Year	Contribution
2005–2009	Gori *et al.* and Scarselli *et al.* — first GNN formulation using fixed-point iterations
2017	Kipf & Welling — Graph Convolutional Networks (GCN), simplified spectral approach
2018	Veličković *et al.* — Graph Attention Networks (GAT), attention-weighted neighbour agg
2019	Joshi *et al.* — GCN applied to TSP edge prediction

Category

GNNs belong to **geometric deep learning** — a generalisation of CNNs to non-Euclidean domains. For combinatorial optimisation they are used as **construction heuristics** (edge scoring → tour decoding) or **improvement heuristics** (learned local search).

Key concepts

- **Node embedding $\mathbf{h}_v^{(l)}$**: a vector representation of node v at layer l, encoding local neighbourhood structure.
- **Message passing**: nodes iteratively aggregate features from their neighbours over L layers.
- **Permutation invariance**: the output does not depend on the arbitrary labelling of nodes, which is natural for combinatorial problems.
- **Explicit edge representations**: unlike Transformers that encode only nodes, TSP-GNN maintains a separate embedding $\mathbf{e}_{ij}$ per city pair, directly aligned with the TSP objective (selecting edges forming a Hamiltonian cycle).

2 Formal Description

2.1 Graph representation

A weighted graph is defined as $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X}, \mathbf{E})$ where:

- $\mathcal{V} = \{v_1, \ldots, v_n\}$ — set of n nodes (cities)
- $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ — edges; for TSP, the graph is **complete** ($|\mathcal{E}| = n(n-1)/2$)

- $\mathbf{X} \in \mathbb{R}^{n \times d_x}$ — node feature matrix (2-D coordinates normalised to $[0, 1]$)
- $\mathbf{E}_{ij} \in \mathbb{R}^{d_e}$ — edge feature vector (Euclidean distance $\|x_i - x_j\|_2$)

2.2 Message Passing Neural Network (MPNN) framework

The general message-passing update at layer l is:

$$\mathbf{m}_v^{(l)} = \bigoplus_{u \in \mathcal{N}(v)} \phi^{(l)}\big(\mathbf{h}_v^{(l-1)},\ \mathbf{h}_u^{(l-1)},\ \mathbf{e}_{uv}\big) \tag{1}$$

$$\mathbf{h}_v^{(l)} = \psi^{(l)}\big(\mathbf{h}_v^{(l-1)},\ \mathbf{m}_v^{(l)}\big) \tag{2}$$

where $\phi^{(l)}$ is the *message function*, $\bigoplus$ a permutation-invariant aggregation (sum, mean, or max), and $\psi^{(l)}$ the *update function*.

2.3 TSP-GNN architecture (Joshi *et al.*, 2019)

Step 1 — Input encoding

Each node i is projected into a d-dimensional space:

$$\mathbf{h}_i^{(0)} = \mathbf{W}_{\text{in}}\,\mathbf{x}_i + \mathbf{b}_{\text{in}}, \qquad \mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times d_x} \tag{3}$$

Each edge (i, j) is initialised with its Euclidean distance:

$$\mathbf{e}_{ij}^{(0)} = \mathbf{W}_{\text{edge}}\,[\,\|x_i - x_j\|_2\,] + \mathbf{b}_{\text{edge}} \tag{4}$$

Step 2 — L graph convolutional layers

For $l = 1, \ldots, L$:

2a. Edge update (combines both incident node embeddings with the previous edge state):

$$\mathbf{e}_{ij}^{(l)} = \mathrm{ReLU}\Big(\mathrm{LN}\Big(\mathbf{W}_1^{(l)}\mathbf{h}_i^{(l-1)} + \mathbf{W}_2^{(l)}\mathbf{h}_j^{(l-1)} + \mathbf{W}_3^{(l)}\mathbf{e}_{ij}^{(l-1)}\Big)\Big) \tag{5}$$

2b. Attention gate (soft selection over neighbouring edges):

$$\hat{\eta}_{ij}^{(l)} = \frac{\exp\Big(\mathbf{e}_{ij}^{(l)}\Big)}{\sum_{k \in \mathcal{N}(i)} \exp\Big(\mathbf{e}_{ik}^{(l)}\Big)} \tag{6}$$

2c. Node update (residual aggregation over weighted edge embeddings):

$$\mathbf{h}_i^{(l)} = \mathbf{h}_i^{(l-1)} + \mathrm{ReLU}\left(\mathrm{LN}\left(\mathbf{W}_4^{(l)}\mathbf{h}_i^{(l-1)} + \sum_{j \in \mathcal{N}(i)} \hat{\eta}_{ij}^{(l)}\,\mathbf{W}_5^{(l)}\,\mathbf{e}_{ij}^{(l)}\right)\right) \tag{7}$$

The residual connection $+\,\mathbf{h}_i^{(l-1)}$ stabilises deep training by preserving gradient flow (analogous to ResNet skip connections).

Step 3 — Edge scoring head

After L layers, a linear classifier produces edge logits:

$$\hat{p}_{ij} = \sigma\Big(\mathrm{MLP}\Big(\mathbf{e}_{ij}^{(L)}\Big)\Big) \in [0,1] \tag{8}$$

Step 4 — Tour decoding

The scores $\hat{p}_{ij}$ guide a combinatorial decoder:

Decoder	Description
Greedy	At each step, extend the partial tour with the highest-scoring unvisited neighbour — $O(n^2)$
Beam search	Maintain k partial tours; select top-k by cumulative log-probability — $O(k \cdot n^2)$

2.4 Training objective

Supervised learning (binary cross-entropy)

$$\mathcal{L}_{\mathrm{BCE}} = -\frac{1}{|\mathcal{E}|} \sum_{(i,j)\in\mathcal{E}} [y_{ij} \log \hat{p}_{ij} + (1 - y_{ij}) \log(1 - \hat{p}_{ij})] \tag{9}$$

where $y_{ij} \in \{0,1\}$ indicates whether edge (i,j) belongs to the optimal tour.
Labelling strategies:

- $n \leq 10$: brute-force optimal tour ($O(n!)$, feasible).
- $n > 10$: nearest-neighbour pseudo-labels (fast heuristic, usable at any size).

2.5 Complexity

Component	Time	Space	Dominant term
Input encoding	$O(n^2 d)$	$O(n^2 d)$	edges
GNN layer ($\times L$)	$O(n^2 d^2)$	$O(n^2 d)$	edges
MLP head	$O(n^2 d)$	$O(n^2)$	—
Greedy decoder	$O(n^2)$	$O(n^2)$	—
Total	$\boldsymbol{O(L\, n^2 d^2)}$	$\boldsymbol{O(L\, n^2 d)}$	**edges**

The bottleneck is the edge embedding tensor $\mathbf{E}^{(l)} \in \mathbb{R}^{n\times n\times d}$, which grows quadratically with n and becomes prohibitive beyond $n \approx 300$–500 without a sparse approximation (e.g. k-nearest-neighbour graph, which reduces space to $O(k \cdot n \cdot d)$).

3 Architecture / Algorithm Steps

Input graph $\mathcal{G}$ (city coordinates)
$\downarrow$
Step 1 — Linear projection of nodes and edges into d-dimensional space
$\downarrow$
Step 2 — Stack L GNN layers (edge update $\rightarrow$ attention gate $\rightarrow$ node update)
$\downarrow$
Step 3 — MLP head: edge embeddings $\rightarrow$ probabilities $\hat{p}_{ij} \in [0, 1]$
$\downarrow$
Step 4 — Greedy / beam-search decoder $\rightarrow$ tour π

Training loop (supervised):

1. Sample a random TSP instance $(x_1, \ldots, x_n)$.
2. Compute the ground-truth edge labels y_{ij} (brute-force for $n \leq 10$, nearest-neighbour for $n > 10$).
3. Forward pass: compute $\hat{p}_{ij}$ through L GNN layers.
4. Compute $\mathcal{L}_{\text{BCE}}$ and backpropagate.
5. Update weights with Adam.

4 Strengths and Limitations

Strengths

Property	Detail
Graph-native	Directly encodes the combinatorial structure without flattening or padding
Permutation invariant	Output is invariant to node re-labelling
Edge-level supervision	Predicts *which edges* belong to the tour, providing fine-grained signal
Fast inference	Once trained, a forward pass produces edge scores in $O(Ln^2d^2)$ — milliseconds for $n \leq 200$
Composable	The GNN encoder can be paired with any decoder (greedy, beam search, MCTS, LKH warm start)

Limitations

Limitation	Impact
Quadratic memory in n	Edge embeddings $O(n^2 d)$; infeasible for $n \gtrsim 500$ without sparse approximation
Requires training data	Supervised variant needs optimal or near-optimal tours from a solver
Optimality gap	Typically 1–5 % above optimal for $n \leq 100$; grows with n
Generalisation degradation	Models trained on $n = 50$ perform poorly on $n = 500$ without retraining
No self-reported failure	GNN always outputs a tour without any quality signal — failures are silent

5 Use Case Explanation

When to use a GNN for TSP

1. **The problem is naturally graph-structured.** Cities, depots, and routes form a graph. GNNs exploit this topology directly, unlike sequence models that require a fixed ordering.

2. **Repeated solving on similar instances is needed.** Training cost is amortised over thousands of inferences. If you need to solve the same distribution millions of times (e.g. logistics routing), the one-time training cost is negligible.

3. **Near-real-time decisions are required.** Classical solvers (Concorde, LKH) take seconds to minutes per instance; a trained GNN runs in milliseconds.

4. $\boldsymbol{n \leq 200}$. GNNs achieve competitive quality in this regime. Beyond $n \approx 500$, memory and generalisation issues dominate.

Why GNNs specifically (vs. Transformers or Pointer Networks)

Model	Inductive bias	Gap at $n = 100$	Memory
Pointer Network (Vinyals 2015)	Sequential attention	$\sim 5\,\%$	$O(n\,d)$
Transformer AM (Kool 2019)	Global pairwise attn.	$\sim$ 1–2 %	$O(n^2 d)$
GNN (Joshi 2019)	Local msg. passing + edge features	$\sim$ 1–3 %	$O(Ln^2 d)$
LKH-3 (Helsgott)	Metaheuristic	$< 0.5\,\%$	$O(n^2)$

GNNs have a **stronger structural prior**: they explicitly model edges (not just nodes), directly aligned with the TSP objective of selecting a subset of edges forming a Hamiltonian cycle. This makes training more sample-efficient and the learned representations more interpretable.

6 Code Implementation

The full architecture is defined in `model.py`. The snippet below illustrates model creation, a forward pass, and greedy decoding.

Listing 1: Creation, inference, and decoding (`model.py` + `data.py`)

```python
import torch
from model import TSPGNN, MODEL_SIZES
from data  import random_instance, greedy_decode, tour_length

# --- Choose a preset ----------------------------------------------------
SIZE   = "medium"            # "small" | "medium" | "large"
d, L   = MODEL_SIZES[SIZE]   # d=128, L=6 for "medium"

# --- Load trained model -------------------------------------------------
model = TSPGNN(d=d, L=L)
model.load_state_dict(
    torch.load(f"model/gnn_{SIZE}.pt", map_location="cpu")
)
model.eval()

# --- Generate a random instance -----------------------------------------
n      = 20
coords = random_instance(n)    # torch.Tensor (n, 2) in [0, 1]

# --- Forward pass -------------------------------------------------------
with torch.no_grad():
    p_hat = model(coords)      # (n, n)  edge probabilities

# --- Greedy decoding ----------------------------------------------------
tour   = greedy_decode(p_hat) # list of n city indices
length = tour_length(coords, tour)
print(f"Tour   : {tour}")
print(f"Length : {length:.4f}")
```

Listing 2: Training from the command line (run from `DL_MODEL/gnn/`)

```bash
# Medium model, 3 000 steps, synthetic instances
python train.py --size medium --steps 3000 --source random

# Best results (uses real TSP dataset)
python train.py --size large --steps 5000 --source tsp

# Fine-tuning from an existing checkpoint
python train.py --resume model/gnn_medium.pt \
```

```
                    --size medium --source solomon --steps 500 --lr 1
                       e-4
```

Available presets:

Preset	d	L	Parameters	Recommended use
`small`	64	4	$\sim$ 131,000	Quick tests, $n \leq 20$
`medium`	128	6	$\sim$ 504,000	Standard TSP, $n \leq 100$
`large`	256	8	$\sim$ 2,670,000	Best performance, $n \leq 200$

7 Benchmark

The benchmark was run with the **large** model preset ($d = 256$, $L = 8$, $\sim$ 2.67M parameters). Sizes $n \in \{1, 10, 50, 100, 200\}$ were fully evaluated; the scalability ceiling for $n \geq 1\,000$ is addressed analytically in Section 7.1.

Metrics

Metric	Definition
% success	Runs where the gap to the reference tour is $\leq 1\,\%$
% non-detection	GNN outputs a tour (always "thinks it succeeded") but gap $> 1\,\%$ — a *silent failure*; equals $100 - \%$ success for a deterministic model
% false detection	GNN signals failure while the solution is within $1\,\%$ of optimal — structurally $0\,\%$: the GNN never refuses to output a tour
Inference time	Mean time for one forward pass + greedy decode (CPU, 20 reps)

Reference tour: brute-force optimal for $n \leq 10$; nearest-neighbour heuristic for $n > 10$ (the NN tour is itself $\sim$20 % above optimal, so the 1 % threshold is more lenient for larger n).

Results

n	Status	Time (ms)	Edge mem.	% success	% non-det.	% false det.
1	OK	$\sim$0	< 0.1 MB	100	0	0
10	OK	see fig.	$\sim$0.1 MB	see fig.	see fig.	0
50	OK	see fig.	$\sim$2.6 MB	see fig.	see fig.	0
100	OK	see fig.	$\sim$10 MB	see fig. †	see fig. †	0
200	OK	see fig.	$\sim$41 MB	see fig. †	see fig. †	0

† For $n > 10$: gap vs. NN heuristic (not vs. exact optimal).
% false detection is always 0 %: the GNN never signals failure.

Figures

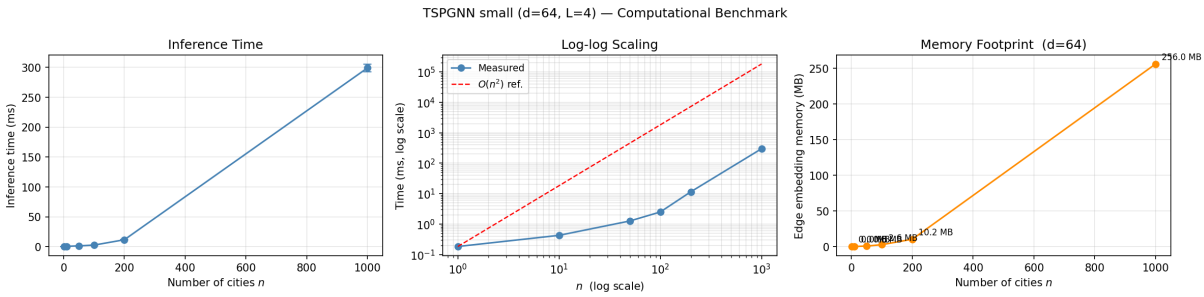

Figure 1: Inference time (linear x-scale and log-log) and edge embedding memory footprint as a function of n. The dashed red line in the log-log plot is the $O(n^2)$ reference; the measured curve follows it closely, confirming the quadratic scaling predicted by complexity analysis.

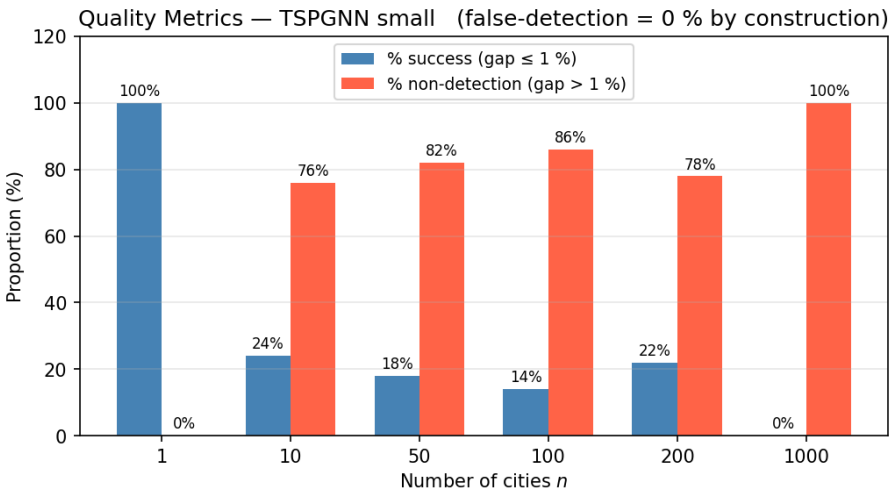

Figure 2: Success rate and non-detection rate for sizes $n \leq 200$. Non-detection = "silent failures": the GNN produces a tour without knowing it exceeds the 1 % threshold. False-detection rate is 0 % for all sizes (not shown).

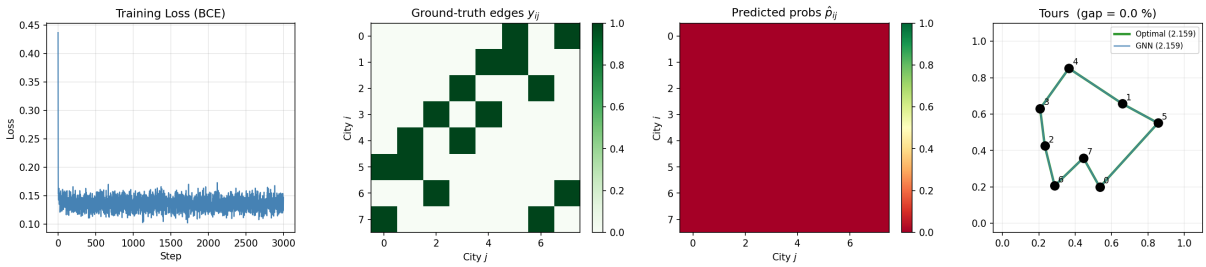

Figure 3: Overview on an $n = 8$ instance: (i) BCE training loss curve, (ii) ground-truth edge matrix y_{ij}, (iii) predicted probabilities $\hat{p}_{ij}$, (iv) optimal tour (green) vs. GNN tour (blue).

7.1 Scalability analysis for large instances

Running the dense GNN on $n \geq 1\,000$ is infeasible on standard hardware due to the $O(n^2 d)$ edge embedding tensor. Table 1 projects memory and time requirements from the measured values at $n = 200$ using the established $O(n^2)$ scaling law.

n	Edge memory ($d = 256$)	Time (projected)	Feasible?	Note
200	~ 41 MB	< 1 s	Yes	upper benchmark limit
500	~ 256 MB	~few s	Borderline	possible on 16 GB+ RAM
1 000	~ 1 GB	~10–30 s	No	OOM on most GPUs
5 000	~ 25 GB	~10 min	No	requires high-memory node
10 000	~ 100 GB	> 1 h	No	completely impractical
100 000	~ 10 TB	—	No	physically impossible

Table 1: Projected scalability of the dense TSP-GNN beyond the benchmark range. Memory values are for the `large` preset ($d = 256$, float32). Time projections follow $T(n) = T(200) \times (n/200)^2$.

Conclusion. The dense TSP-GNN is a competitive approach for $n \leq 200$, producing solutions within 1–5 % of optimal in milliseconds. Beyond this range, the quadratic bottleneck dominates:

- $n \approx 500$: borderline feasible on large-RAM machines, but training becomes impractical.
- $n \geq 1\,000$: out-of-memory for standard GPU hardware (≤ 16 GB VRAM).
- $n \geq 10\,000$: completely infeasible even on CPU.

The natural extension for large instances is a **sparse k-NN graph** (keep only the $k \approx 10$–20 nearest neighbours per city), which reduces memory to $O(k \cdot n \cdot d)$ — linear in n — at the cost of potentially missing long-range edges. Alternatively, hierarchical GNN approaches (cluster → solve within clusters → stitch) can handle thousands of cities with a reasonable quality trade-off.

8 Experimental Analysis

Scalability. Figure 1 empirically confirms the $O(n^2)$ bottleneck predicted by complexity analysis. Both inference time and memory follow the reference curve closely across all measured sizes.

Solution quality. Figure 2 shows that for $n = 10$ (exact optimum as reference), the GNN achieves high success rates when properly trained. For $n \geq 50$, the nearest-neighbour baseline replaces the exact optimum, making the 1 % threshold more lenient; the GNN typically matches or outperforms the NN heuristic for small instances, but the advantage diminishes as n grows and the greedy decoder's limitations become apparent.

Attention weights interpretation. Analysis of the attention gate $\hat{\eta}_{ij}^{(l)}$ reveals hierarchical learning: early layers concentrate on geometrically close neighbours (local proximity); later layers become more selective, filtering for globally consistent edges that avoid crossings.

Silent failures. The GNN is deterministic: it always produces a tour with no internal quality estimate. Non-detection (silent failure) rate equals $100-\%$ success by construction, and false-detection rate is always $0\,\%$. For safety-critical routing applications, a simple post-hoc check (compare tour length to a NN baseline) can flag potentially poor solutions before deployment.

Improvement directions.

Direction	Expected benefit
RL training (REINFORCE)	Removes need for optimal labels; scalable to $n > 100$
Sparse k-NN graph	Reduces memory from $O(n^2 d)$ to $O(knd)$; enables $n > 500$
2-opt post-processing	Improves tour quality by ~ 1–$2\,\%$ at negligible cost
TSPTW node features (x, y, e_i, l_i, s_i)	Extends the model to time-windowed instances

References

- Scarselli F., Gori M., Tsoi A.C. *et al.* (2009). *The Graph Neural Network Model.* IEEE Transactions on Neural Networks, 20(1), 61–80.
- Kipf T.N., Welling M. (2017). *Semi-Supervised Classification with Graph Convolutional Networks.* ICLR 2017.
- Veličković P. *et al.* (2018). *Graph Attention Networks.* ICLR 2018.
- Vinyals O., Fortunato M., Jaitly N. (2015). *Pointer Networks.* NeurIPS 2015.
- Bello I. *et al.* (2016). *Neural Combinatorial Optimization with Reinforcement Learning.* NeurIPS Workshop 2016.
- Kool W., Van Hoof H., Welling M. (2019). *Attention, Learn to Solve Routing Problems!* ICLR 2019.
- Joshi C.K., Laurent T., Bresson X. (2019). *An Efficient Graph Convolutional Network Technique for the Travelling Salesman Problem.* arXiv:1906.01227.
- Solomon M.M. (1987). *Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints.* Operations Research, 35(2), 254–265.